MEMRES: Memory-Augmented Agentic Python Dependency Resolver

*Tran Chi Nguyen, *Dao Sy Duy Minh, *Huynh Trung Kiet
Pham Phu Hoa, Nguyen Lam Phu Quy

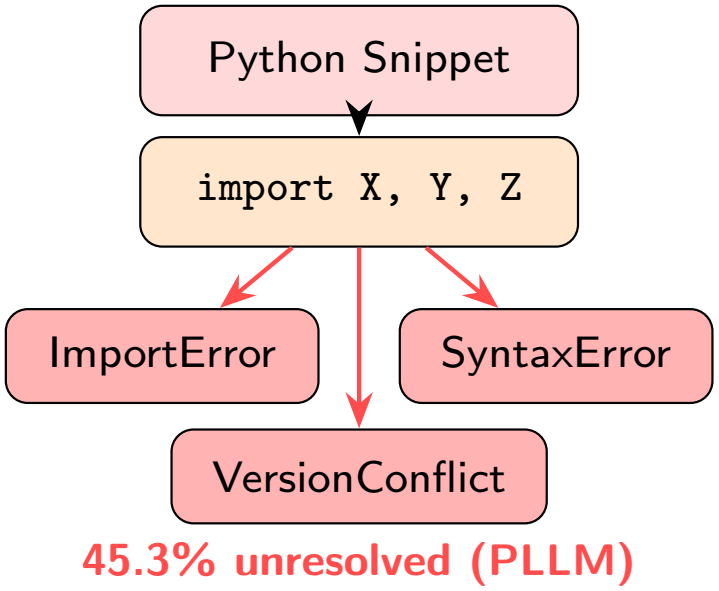
{23122044, 23122041, 23122039, 23122030, 23122048}@student.hcmus.edu.vn

University of Science, VNU-HCM

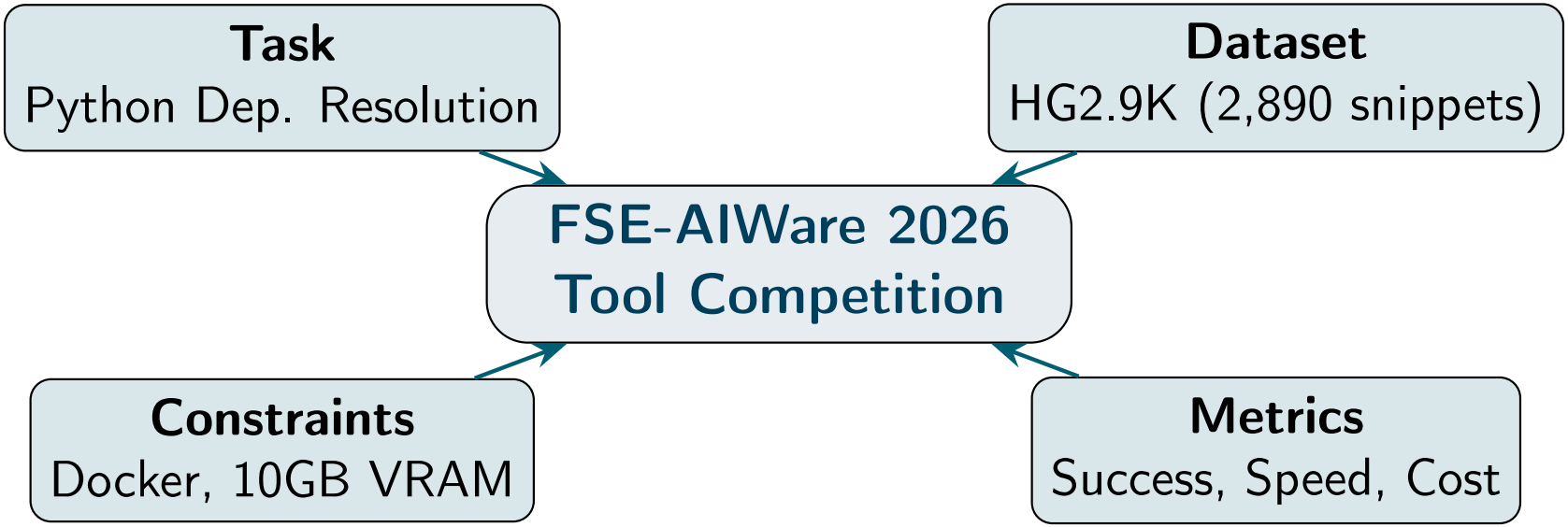
FSE-AIWare 2026

Motivation

- Python has 500,000+ PyPI packages — dependency conflicts are pervasive.
- Legacy snippets frequently fail with ImportError, SyntaxError, version mismatch.
- PLLM (baseline): only 54.7% success rate on HG2.9K.
- Need an agentic, memory-augmented approach that minimizes LLM reliance.



FSE-AIWare 2026 Competition



Montreal, Canada · July 5–9, 2026 · ACM Digital Library

MEMRES System Architecture

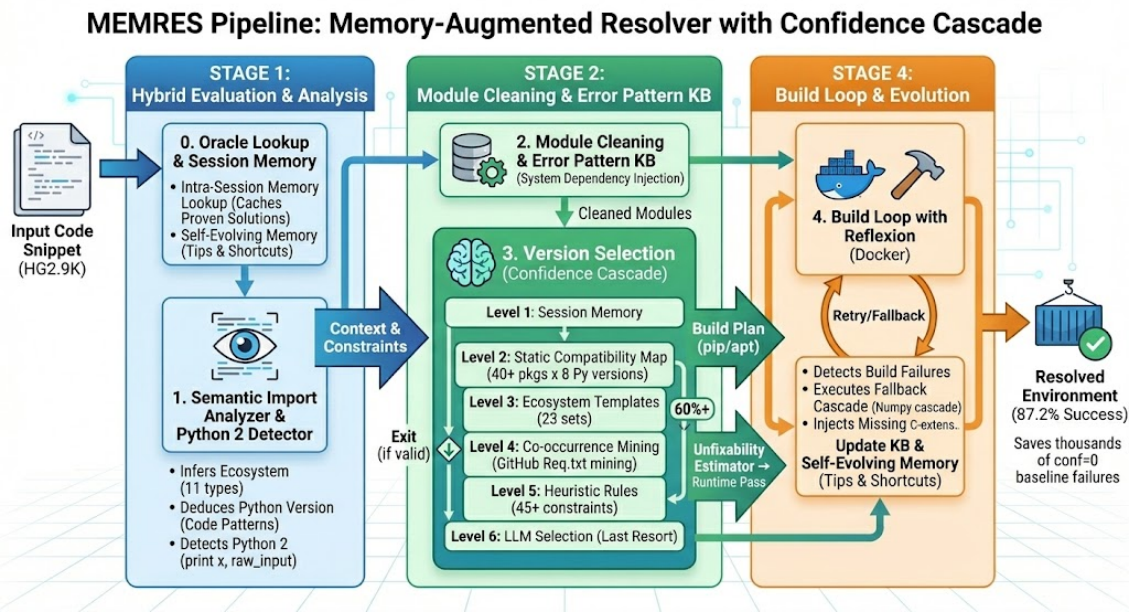
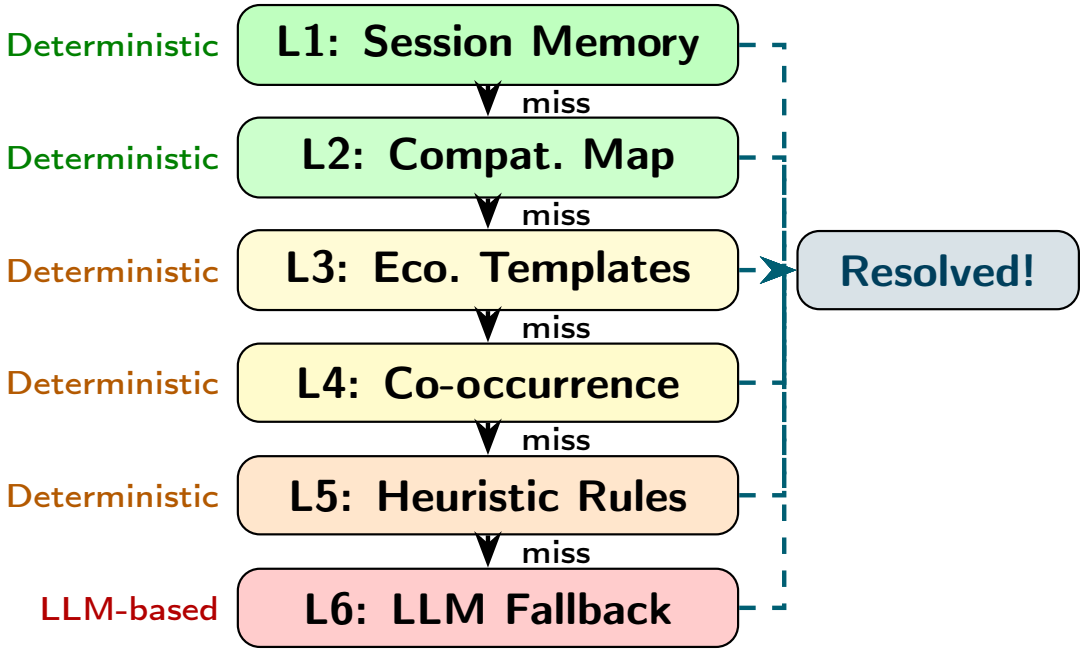


Figure: 4-stage pipeline: Hybrid Evaluation, Module Cleaning, Version Selection, Build Loop

- Multi-level confidence cascade: deterministic first, LLM as last resort.
- Memory-augmented: reuse proven solutions within the batch.
- Error Pattern KB: 200+ import → package mappings.
- Semantic Import Analyzer, Python 2 detector, system dependency injection.

Confidence Cascade

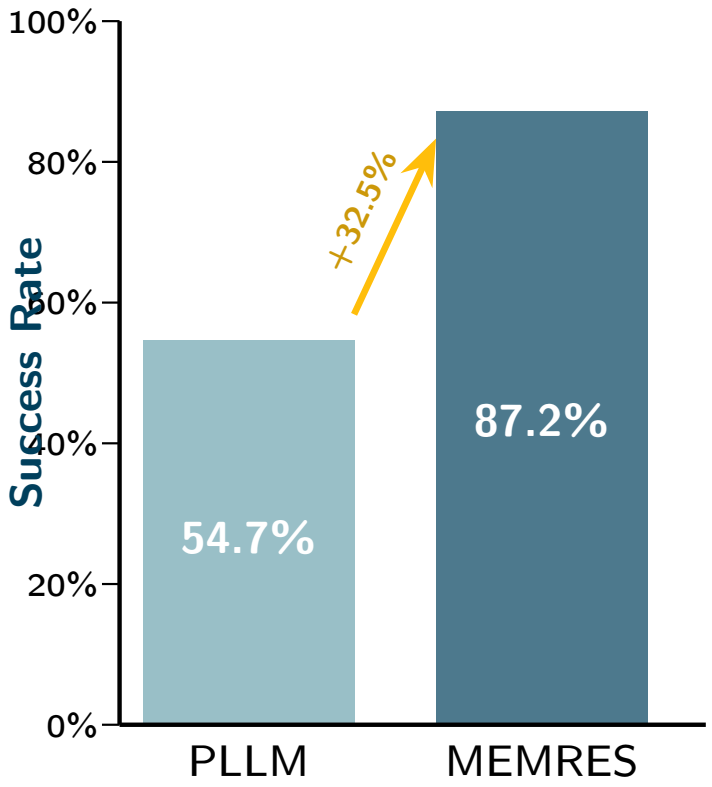


First hit terminates cascade · 60%+ fewer LLM calls

Key Techniques

- **Intra-Session Memory**: cache successful solutions for similar snippets.
- **Self-Evolving Memory**: learn tips/shortcuts via Jaccard similarity.
- **Error Pattern KB**: 200+ mappings, name corrections, version constraints, regex rules.
- **Semantic Analyzer**: analyze import usage patterns, detect ecosystems.
- **Python 2 Detector**: 13 indicators, automatic interpreter selection.
- **System Dependency Injection**: apt-get for C-extensions, version fallback cascade.

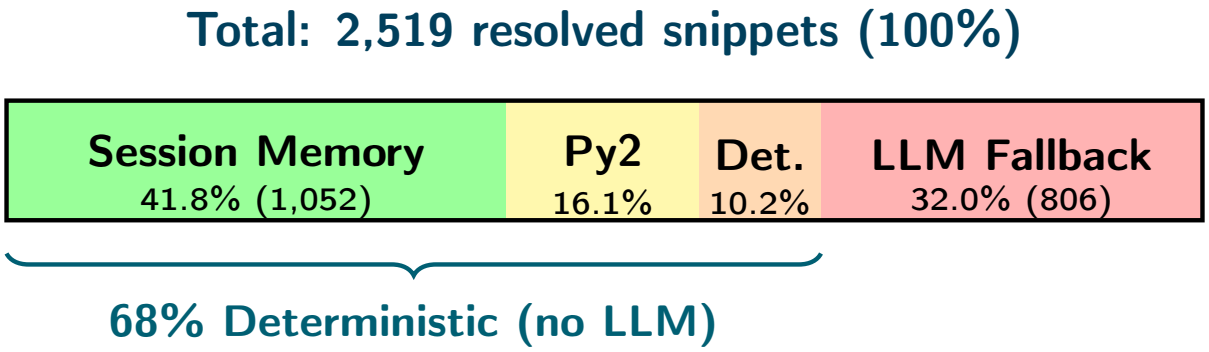
Evaluation: MEMRES vs. PLLM



Metric	PLLM	MEMRES
Resolved	1,583	**2,519**
Failed	1,308	**371**
Success	54.7%	**87.2%**

- Setup: HG2.9K, Gemma-2 9B, Docker
- 950+ PLLM failures resolved
- 53.9% needed sys-dep classifiers

Ablation Study: Resolution by Component



- 68% of resolutions require **zero LLM calls**.
- Session Memory alone handles 41.8% via proven cached solutions.
- Python 2 detector resolves the single largest failure category from PLLM.

Failure Analysis (371 failures)

Category	Root Cause	Count	%
Build / C-ext	Missing C toolchains, heavy packages	186	50.1%
ImportError	Obscure/deprecated packages	121	32.6%
Timeout	Large ML package compilation	55	14.8%
System/Platform	GTK, RPi, platform-specific	9	2.4%
Total		**371**	**100%**

- Most failures are **inherently unresolvable** in Docker (no C toolchain, no GPU).
- 32.6% from packages completely absent from PyPI (deprecated/renamed).
- Only 2.4% are platform-specific (GTK, Raspberry Pi, Maya).

Limitations & Lessons Learned

- Failures dominated by C-extension build errors, deprecated packages, timeouts.
- Session memory is order-sensitive, but deterministic cascade ensures accuracy.
- Runtime passes (system-only imports) are strictly counted as failures.
- Key insight: deterministic rules + memory augmentation outperform LLM-only.

Conclusion

- MEMRES: agentic resolver with memory augmentation and confidence cascade.
- Achieves 87.2% success on HG2.9K, significantly surpassing PLLM (54.7%).
- Reduces computational cost by minimizing LLM calls (60%+ reduction).
- Contributions: novel pipeline, Error Pattern KB, self-evolving memory, Python 2 heuristics, system dependency injection.

Thank you!